

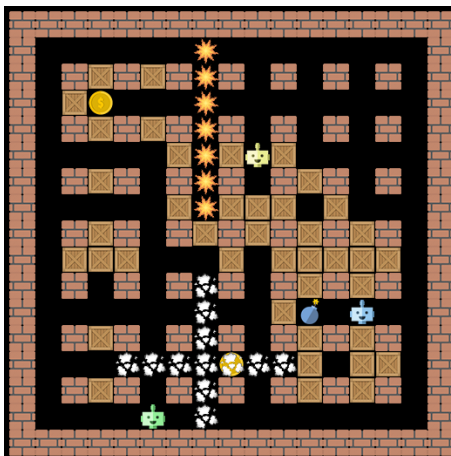
Final Project

Reinforcement Learning for Bomberman

Deadline for agent code: Mon, 21.09.2026, 21:00.

Deadline for report: Mon, 28.09.2026, 21:00.

Ask questions on Discord to #final-project-questions



In the final project you will use reinforcement learning techniques to train an agent to play the classic arcade game Bomberman.

In our setting, the game is played simultaneously by four agents in discrete time steps. Your agent can move around, drop bombs or stand still. Crates can be cleared by well-placed bombs and will sometimes uncover coins, which you can collect for points. The deciding factor for the final score, however, is to blow up opposing agents, and to avoid getting blown up yourself. To keep things simple, special items and power-ups are not available in our version of the game.

After the project deadline, we will hold a tournament between all trained agents with real prizes. Tournament performance will be a factor in the final grade, although the quality of your report and a systematic (scientific) approach to agent design, optimization, and testing will carry much more weight (see section 9 for detailed instructions).

You should develop two (or more) different models and submit your best performing model to the tournament. All models should be described in the report. However, do **not** split labor such that every team member works on their separate model – we attach great importance to real teamwork!

This is an open project. You may use any combination of models and techniques you have learnt during the semester. You are also free to extend your knowledge to new methods and use other/more advanced models and techniques, although we request that at least one of your models focuses on techniques from the lecture. There is only one strict rule: **Your solution must involve machine learning or it will be rejected.**

1 Regulations

Use of AI

It is permitted to consult an AI to brainstorm ideas and to help writing the code and drafting the report. However, it must be clear in the code and report that the main work was done by yourselves. **If you do not refine AI drafts in your own style, you will fail the examination.**

Submission

Your first submission (agent code) for the final project should consist of the following:

- A directory containing the agent code of your best performing model, including all trained parameters. This is the subdirectory of `agent_code` that you are developing your agent in (see details below).

Zip all files into a single archive `final-project-agent-code.zip` and upload this file to MaMPF before the given deadline.

Moreover, please set your **Anzeigename/display name** and **Name in Uebungsgruppen/name in tutorials** in MaMPF to your real name, which should be identical to your name in `muesli` and make sure you **join the submission** of your team via the invitation code before the submission deadline. Check out <https://mampf.blog/handing-in-homework-assignments> for instructions.

Your second submission (report) for the final project should consist of the following:

- A PDF report of your approach. The report should comprise about **4000 words per team member** (and please not much more) and – for legal reasons – indicate after headings who is responsible for each subsection.
- The URL of a public repository containing your entire code base (including all models you developed), which must be mentioned in the report. Please **do not upload your report** to this repository.

You are allowed to switch groups between the homework assignments and this final project. Please organize yourself, e.g. via `#homework-team-finding`, and announce your teams (including a catchy team name for the tournament) at <https://tinyurl.com/fml-final-project-teams>.

It goes without saying that cheating (e.g. resubmission of a solution from a previous edition, even if slightly revised) is forbidden and will result in a failed grade. Tutors are quite good at detecting these attempts.

Development

To share resources fairly between competing agents, you are not allowed to use multiprocessing in your final agent. However, multiprocessing or anything else that comes to your mind to improve *training* is perfectly fine.

If you choose to learn and use neural networks, this is okay. Take into account that neural networks will be executed on the CPU during official games, although you may use GPUs during training. Also beware that your agent may not be competitive when network training takes longer than expected and has not converged by the deadline.

Playing around with the provided framework is allowed, and may in fact be necessary to facilitate fast training of your agent. However, be aware that the final agent code will simply be plugged into our *original* version of the framework – any changes you made to other parts will not be present during official games.

Discussions about the final project with other teams are very much encouraged. You can share your trained agents (*without training code*) on `#final-project-beat-my-agent` and download other teams' agents to test your approach. Just keep in mind that in the tournament you will compete for prizes, so you may want to keep your best ideas to yourself :)

The lectures on July 14 to 16 explained the necessary basics of reinforcement learning. You may also watch video recordings of the lecture from previous years on MaMPF. In addition, the internet provides plenty of material on all aspects of RL. Study some of it to learn more and get inspiration. The use of free software libraries (e.g. `pytorch`) is allowed, if they can be installed from an official repository like `pip` or `conda`, but you must not copy-paste any existing solution in whole or in part. Plagiarism will lead to a “failed” grade.

2 Setup

You can find the framework for this project on [Github](#):

```
git clone https://github.com/ukoethe/bombberman_rl
```

It contains:

- the game environment with all APIs you need for reinforcement learning,
- rule-based agents of different strength and a very weak random agents (all those cannot learn),
- a template for your own agent.

Direct any questions about the framework to [#final-project-questions](#) in Discord. If you find a bug in the game environment, consider submitting a Github issue or pull request.

To show the game in a user interface, we are using the `pygame` package. If you do not install `pygame`, the game will still run, but without the GUI. It is also recommended to install the `tqdm` package which gives you feedback how fast the rounds are simulated. You can install both packages in your existing `conda` environment from the exercises:

```
conda activate ml_homework
pip install pygame tqdm
```

Other than that, we assume a standard Python 3 installation with `numpy`, `scipy` and `sklearn`. Clearly indicate in your repository and at the beginning of your report which additional libraries we need to install.

You are now ready to watch a few rounds of Bomberman as played by our strong rule-based agent as follows:

```
cd bombberman_rl
python main.py play
```

3 General setting and rules of the game

The game is played in discrete steps by one to four agents, who are represented by robot avatars in different colors. Because of the random elements, multiple episodes of the game will be played to determine a winner by total score.

In one step of a game episode, each robot can either move one tile horizontally or vertically, drop a bomb or wait. Movement is restricted to empty (i.e. black) tiles – stone walls, crates, bombs and other agents block movement. Coins are an exception, as they are collected when moving onto their tile. While the placement of stone walls is the same for every round, the distribution of crates and the hiding places of coins differ each time. Agents always start in one of the board's corners, but it is randomly determined which.

Once a bomb is dropped, it will detonate after four steps and create an explosion that extends three tiles up, down, left and right (so you can just outrun the explosion). The explosion destroys crates and agents, but will stop at stone walls and does not reach around corners. The explosion remains dangerous for one more round before it vanishes in smoke. The smoke is not dangerous. Agents can drop a new bomb after the explosion is gone.

A fixed number of coins is hidden at random positions in each episode. When the crate concealing a coin is blown up, the coin becomes visible and can be collected. Collecting a coin gains the agent one point, while blowing up an opponent is worth five points.

Every episode ends after 400 steps. There is a fixed time limit of 0.5 seconds¹ per step for agents to arrive at their decisions. Agents exceeding this time will not execute the action picked, but stand

¹ Assume that your agent will have exclusive access to one thread of an AMD Ryzen 5 2600 processor and can use up to 8 GB of RAM when we run the tournament.

still as if they had returned 'WAIT'. In the subsequent step, the thinking time is reduced by the time excess in the previous step.

The exact numbers used for all these rules can be found in `settings.py`. They may be subject to change until seven days before the deadline, in case you inform us about major problems with the current settings. You will be notified if any of the rules are adapted.

4 Tasks your agents will have to solve

The most important factor for your final grade is that you conduct the design, optimization, and testing of your agent in a systematic way according to the scientific method. This means that the task should be divided into more manageable subgoals, and each stage should involve meaningful experiments that compare different variants of your agent among each other and with the agents provided by us. The experiments should demonstrate how your agents get better after each modification suggested by the previous round of testing. Setting up these experiments, defining good performance metrics, and describing everything properly in the report are crucial for a good grade.

To get you started at defining and attaining subgoals, we define preliminary tasks that help your agent to evolve from simple to complex and ease debugging. The tasks are subsets of each other, so an agent that can handle task 4 should also be able to solve 1, 2 and 3. Configuration instructions for these tasks are given in section 6 with examples in section 7. Feel free to refine the task subdivision as you see fit.

1. On a game board without any crates or opponents, collect a number of revealed coins as quickly as possible. This task does not require dropping any bombs. The agent should learn how to navigate the board efficiently.
2. On a game board with randomly placed crates yet without opponents, find all hidden coins and collect them within the step limit. The agent must drop bombs to destroy the crates. It should learn how to use bombs without killing itself, while not forgetting efficient navigation. Escaping bombs is a crucial capability for good tournament performance, so place proper emphasis on this step.
3. On a game board with crates, try to hunt and blow up our predefined `peaceful_agent` (easy) and the `coin_collector_agent` (hard). The former agent does not drop bombs and moves randomly. The latter agent will drop bombs only for the purpose of collecting coins.
4. On a game board with crates, hold your own against one or more opposing agents (e.g. the full-strength `rule_based_agent`, different variants of your own design) and fight for the highest score. Experience shows that you must be able to beat the `rule_based_agent` in order to have any chance of winning the tournament.

Note that you can manually change some of the game parameters such as board size to adjust the difficulty of the tasks in the early stages of experimentation, see section 6. You may use reward shaping (see section 7) to guide the agent towards desirable game states, but be aware that your auxiliary rewards are not present in official games, so avoid overfitting to them. Remember that a systematic approach and careful experimentation are key for a good grade.

5 Framework structure and interface for your agent

In reinforcement learning, we distinguish between the agent and the environment it has to interact with. Let us start with the environment here.

Environment

The game world and logic is defined in `environment.py` in the class `BomberLeWorld`. It keeps track of the board and all game objects, can run a step of the game, start a new round and optionally

keeps a reference to the GUI. What's most interesting for you is that it keeps track of the agents playing the game via objects of the **Agent** class defined in **agents.py**. In addition to tracking position, score etc., each **Agent** loads your code, located in an underlying agent folder (see below).

This part of the code is fixed during the tournament. For training, you are allowed to perform arbitrary changes to help your agent train, e.g. by changing the board.

Agent

The environment calls your agent's code at certain times in the game loop. Your code must be located in a subdirectory of **agent_code**.

Always loaded: callbacks.py

Before running the first game, your evaluation code from a file called **callbacks.py** is imported. This file must contain at least two functions:

```
def setup(self): ...  
def act(self, game_state: dict): ...
```

The function **setup(self)** is called once before the first round starts. This gives you a place to initialize everything you need. The **self** argument is a persistent object that will be passed to all subsequent callbacks as well (treat it like an instance of an object in an object oriented framework). That means you can assign values or objects to it which you may need later on. For example:

```
def setup(self):  
    self.model = MyModel()  
  
def act(self, game_state: dict):  
    return self.model.propose_action(game_state)
```

Note that some fields of **self** are pre-set:

logger : <code>logging.Logger</code>	A logger with which you can print debugging messages (see Section 7).
train : <code>bool</code>	This is set to <code>True</code> when your model should train. For example, you might want to load a previously trained model when self.train is <code>False</code> and else create a fresh model to be trained.

The function **act(self, game_state)** is called once every step to give your agent the opportunity to figure out the best next move. The available actions are 'UP', 'DOWN', 'LEFT', 'RIGHT', 'BOMB' and 'WAIT', with the latter being the default if your code exceeds the time limit. Choose an action by returning the corresponding string from **act(self)** (see above).

The current state of the game world is passed in the dictionary **game_state**. It has the following entries:

<code>'round': int</code>	The number of rounds since launching the environment, starting at 1.
<code>'step': int</code>	The number of steps in the episode so far, starting at 1.
<code>'field': np.array(width, height)</code>	A 2D numpy array describing the tiles of the game board. Its entries are 1 for crates, -1 for stone walls and 0 for free tiles. Note that we use image coordinates (x, y) , so <code>print(game_state['field'])</code> will show the board transposed compared to the GUI.
<code>'bombs': [(int, int), int]</code>	A list of tuples $((x, y), t)$ of coordinates and countdowns for all active bombs (countdown == 0 means that the bomb is about to explode).
<code>'explosion_map': np.array(width, height)</code>	A 2D numpy array stating, for each tile, for how many more steps an explosion will be present. Where there is no explosion, the value is 0.
<code>'coins': [(x, y)]</code>	A list of coordinates (x, y) for all currently collectable coins.
<code>'self': (str, int, bool, (int, int))</code>	A tuple $(n, s, b, (x, y))$ describing your own agent. n is the agent's name, s its current score, b is a boolean indicating whether the 'BOMB' action is possible (i.e. no own bomb is currently ticking), and (x, y) is the coordinate on the field.
<code>'others': [(str, int, bool, (int, int))]</code>	A list of tuples like the one above for all opponents that are still in the game.
<code>'user_input': str None</code>	User input via the GUI. See Section 7 for details.

Loaded only when training: train.py

When set to training mode, the environment additionally imports your training code from a file called `train.py` from the same subdirectory. This file must contain the following additional callbacks:

- `setup_training(self)` is called when loading the agent, after calling `setup` in `callbacks.py`. Use this to initialize variables you only need for training.
- `game_events_occurred(self, old_game_state, self_action, new_game_state, events)` is called once after each step except the last. At this point, all the actions have been executed and their consequences are known. Use this callback to collect training data and fill an experience buffer.
- The other callback is `end_of_round(self, last_game_state, last_action, events)`, which is very similar to the previous, but only called once per agent after the last step of a round.

In any of the last two callbacks, your learning should take place. You have the freedom to choose where you see it more useful. Note that neither of these functions has a time limit.

The `events` parameter will hold a list of game events that happened as a consequence of your and the opposing agents' actions. You can use these as a basis for auxilliary rewards and penalties to speed-up training. They will not be available when the game is run without training mode. All available events are stored in `events.py`:

```
import events as e
```

They are defined as follows:

e.MOVED_LEFT	Successfully moved one tile to the left.
e.MOVED_RIGHT	Successfully moved one tile to the right.
e.MOVED_UP	Successfully moved one tile up.
e.MOVED_DOWN	Successfully moved one tile down.
e.WAITED	Intentionally didn't act at all.
e.INVALID_ACTION	Picked a non-existent action or one that couldn't be executed.
e.BOMB_DROPPED	Successfully dropped a bomb.
e.BOMB_EXPLODED	Own bomb dropped earlier on has exploded.
e.CRATE_DESTROYED	A crate was destroyed by own bomb.
e.COIN_FOUND	A coin has been revealed by own bomb.
e.COIN_COLLECTED	Collected a coin.
e.KILLED_OPPONENT	Blew up an opponent.
e.KILLED_SELF	Blew up self.
e.GOT_KILLED	Got blown up by an opponent's bomb.
e.OPPONENT_ELIMINATED	Opponent got blown up.
e.SURVIVED_ROUND	End of round reached and agent still alive.

These events summarize a lot of the changes between two game states. One key factor in successfully training your agent may be to give useful auxiliary rewards to your agents based on the predefined and your own events.

Summary of callbacks

The following block of code summarizes how the environment calls the agent's callbacks:

```
import callbacks
callbacks.setup(...)

if train:
    import train
    train.setup_training(...)

for round in range(n_rounds):
    ... # Reset world

    for step in range(n_steps):
        callbacks.act(...)
        ... # Perform agents' actions
        ... # Evaluate world: Bombs, explosions
        if train and not dead:
            train.game_events_occurred(...)
    if train:
        train.end_of_round(...)
```

Customize your agent

You can customize your agent's appearance in the GUI by adding files `avatar.png` and `bomb.png` in your agent directory. They should be 30x30 pixels large PNG images and will be used in the award ceremony.

6 Putting it all together

In order to train a new agent, you must create a subdirectory within `agent_code` with your agent's name – this name will also identify your agent during the tournament. It is a good idea to start by copying the `tpl_agent` to your own directory. We choose to name it “my_agent” as an example. Within the new `agent_code/my_agent/`, you should find the files `callbacks.py` and `train.py` as described above. Other custom files, such as trained model parameters, must also be stored in this directory!

Run your agent on the command line:

```
python main.py play --my-agent my_agent
```

Now you should see your newly created agent play against three strong rule-based agents.

If you want to specify more than one agent to run, call

```
python main.py play --agents my_agent random_agent rule_based_agent peaceful_agent
```

to see your agent compete with a random agent, a strong rule-based agent, and a peaceful version of the random agent, which does not drop bombs.

You can also create and run several agents of your own, or even the same agent multiple times, in order to train your agents by self-play. If you choose this strategy, it may be helpful to add a random element to the choice of actions to avoid quick convergence to a bad local optimum.

For training, call the main script with `--train N`: Here, N specifies that the first N agents passed to `--agents` should be trained. If you are using the `--my-agent` parameter, only $N = 0, 1$ run successfully, since the rule based agents cannot be trained.

```
python main.py play --agents my_agent random_agent rule_based_agent  
peaceful_agent --train 1  
# or  
python main.py play --my-agent my_agent --train 1
```

This automatically shortens the game to run until the last training agent has died. You can prevent this behaviour with `--continue-without-training`.

7 Practical hints to get you started

The `self` object passed to your callback functions comes with logging functionality that you can use to monitor and debug what your agent is up to:

```
self.logger.info(f"Choosing an action for step {self.game_state['step']}.")  
self.logger.debug("This is only logged if s.log_agent_code == logging.DEBUG")
```

All logged messages will appear in a file named after your agent in your subdirectory. Continuing with the example from earlier, that would be `agent_code/my_agent/logs/my_agent.log`. If you find yourself spammed with log messages, decrease the log levels in `settings.py`.

`agent_code/rule_based_agent/` contains code for a rule-based agent that plays the game very well. Look at this agent's `act(self)` callback to see an example of how reading the game state, logging and choosing an action are done. You can choose this agent as a training opponent, or even use it to create training data while your own agent is still struggling. However, your own agent must be trained (not rule-based) to fulfill the project requirements.

If you pass `--save_replay`, a new replay file will be saved in `replays/` for each episode. To watch it back, call the command as follows:

```
python main.py replay <stored-replay>
```

Both the `play` and `replay` modes of `main.py` receive several options influencing the interface of the game. Be sure to get an overview by running

```
python main.py [re]play --help
```

To speed up the game, pass `--skip-frames` so that not every step is rendered. For efficient training, you should of course turn off the graphical user interface completely by passing `--no-gui`. The game will then run as fast as the agents can make decisions.

There is also the option to control one of the agents via keyboard in the GUI. To do so, include the agent named `user_agent` and use the arrow keys to move around, `Space` to drop a bomb and `Return` to wait. This is clearly not an efficient way to create training data for your agent, but helps you develop an intuition for the game's behaviour. This might, for example, be useful for designing

auxilliary rewards or learning curricula. It is useful to combine this agent with `--turn-based`. This will wait for a key press until the next action is executed.

If you wish to remove randomness from the game world, you can use the `--seed` flag. Using a fixed seed will result in crates and coins always appearing in the same places. This will not affect the randomness of agents however, which use a separate random state.

For convenience, you can use predefined scenarios to train on Tasks 1-4. For example, you could use the following commands for each task:

1. `python main.py play --no-gui --agents my_agent --train 1 --scenario coin-heaven`
2. `python main.py play --no-gui --agents my_agent --train 1 --scenario classic`
3. `python main.py play --no-gui --agents my_agent peaceful_agent
coin_collector_agent --train 1 --scenario classic`
4. `python main.py play --no-gui --agents my_agent rule_based_agent --train 1
--scenario classic`

Note that the `classic` scenario is the default and is only specified in these examples for clarity. You can add your own scenarios in `settings.py`.

Successful strategies from previous competition winners

In the past, it hasn't always been the most complicated model which has won the competition. Groups have successfully won with simple but carefully designed and trained models. Some things to consider:

Feature Engineering. It is very important to train your model with good features (unless you use neural networks to learn powerful features automatically). Spend time thinking and experimenting about how you can condense the most important information into a low-dimensional vector. Some examples of strong features:

- Situational awareness features, e.g. whether or not there is a wall to the left of your agent.
- Pathfinding features, e.g. the direction to move which brings you closest to the nearest coin.
- Life-saving features, e.g. whether or not the agent is in the path of a bomb which is about to explode.

This list is not conclusive, and the exact way that features are implemented will depend on which type of model you decide to train. Remember that you are not allowed to submit a model which does not learn from the features you define! This would disallow, for example, a feature that deterministically returns "the action which results in the best move".

Deep Learning. This is a very powerful approach to reinforcement learning (especially when it automatically learns the features), and several teams have successfully used it in the past. However, it has also often happened that the model was not converged by the tournament deadline, resulting in unsatisfactory playing strength. If you attempt a deep learning approach, reserve plenty of time for finetuning of your neural network architecture and training.

Reward Shaping. Rewards gained from collecting coins or blowing up opponents are quite sparse and many models will struggle to converge to a solution based on these alone. Your model likely needs a more dense reward signal. Spend time designing a reward and penalty scheme and experiment with changing the values. Start with assigning rewards to the predefined events in Section 5 and then assign rewards to custom events of your own. Some ideas:

- Use rewards which incentivize using the information supplied by the features. If your feature tells you the direction to the nearest coin, give a reward if the agent chooses to move in that direction.

- Use balanced rewards and penalties for opposing actions. For example, if you give a reward when your agent moves towards a coin, give a negative reward of the same (or higher) magnitude for moving away from the coin or waiting. Otherwise you create an incentive to move back and forth to continue collecting the positive movement reward instead of making real progress.
- Don't be afraid to define a large number of custom events and rewards/penalties. The more dense the reward signal, the better (provided that it is carefully designed to be useful). The computational burden of adding rewards is usually not as great as that of adding features.
- Be careful to shape your rewards such that they don't push your agent towards a bad local optimum. Theory says that auxiliary rewards should only depend on the game states and not on the actions leading there.²

Custom environment. Complementary to reward shaping, you can change the game itself. For example, it can be useful to adjust the field size, the amount of crates and coins, the timeout limit, etc. They are defined in and can be accessed from the code after importing `settings.py`:

```
import settings as s
```

As an example, we provide the `coin-heaven` scenario with no crates and more coins. If you change anything, remember that you should test your agent with the original values before submitting it.

Hyperparameter optimization. All machine learning models have hyperparameters, which need to be optimized. Do not neglect this part of the project! Performance of the same model can be dramatically better with the right selection of hyperparameters. Finding them takes time!

Symmetries. The Bomberman game world has both rotational and mirror symmetries. You might be able to exploit these symmetries to make learning faster or more robust. For example, you might identify certain states as being equivalent up to a horizontal flip and therefore requiring the same action, but flipped horizontally. Or you might augment your data set with flipped and rotated versions of a game state.

8 Submission test

To make sure your agent does not throw any errors and is compatible with our setup, we offer (1) a Docker container with the Python environment we will use for the tournament, and (2) pre-runs of your code on our tournament machines. The Docker file is contained on the `bomberman_rl` repository you have already cloned from Github (please `git pull` to get the newest version of the Docker file).

1. Download and install Docker Desktop from <https://docs.docker.com/get-docker/>
2. Run the command `docker run hello-world` to check if the installation was successful
3. In Terminal, navigate to the `bomberman_rl` folder which includes `Dockerfile` and run the command `docker build .` (note the dot). It builds an image equal to the one that we will run your submitted agent code in
4. Run the command `docker images` to check if the image was successfully built and to view its `IMAGE_ID`
5. You can copy your agent's code into the (temporary) container folder by running the command `docker cp <AGENT_CODE_FOLDER> <CONTAINER_ID>:/home/bomberman/agent_code/<AGENT_NAME>` where `<AGENT_CODE_FOLDER>` is the path to your agent's code folder and `<AGENT_NAME>` is the name of your agent (e.g. `my_agent`). This is only needed if you've updated your agent's code after building the image and you don't want to rebuild it every time.

²see "Policy invariance under reward transformations: Theory and application to reward shaping" at <https://people.eecs.berkeley.edu/~russell/papers/icml99-shaping.pdf>.

6. Run the command `docker run <IMAGE_ID>` followed by the regular commands you would use (e.g. `python main.py play`). The full command could look something like this: `docker run ede17t python main.py play --agents rule_based_agent tpl_agent --no-gui`
You are now in an environment where all the packages that we use are installed. Test your agent's code and check if there are any compatibility issues. You can access the log file of the game as follows:
7. Run the game with your agent as in 5.
8. Run the command `docker ps --all` to view the `CONTAINER_ID` where the game was run before
9. Run the command `docker cp <CONTAINER_ID>:/home/bomberman/logs/game.log ./game.log` to copy the log file from the container directory to the current directory and check `game.log`.

A common error is the use of absolute paths to reference files in other directories. You should use relative paths instead.

To participate in the pre-runs on our machines, upload your solution to MaMpf in the format specified in section 1. If you want to use libraries that aren't installed in the Dockerfile by default, you must specify them in a `requirements.txt` file. We will proceed as follows with your uploaded `.zip`:

1. Unzip the submitted file.
2. Install libraries specified in `requirements.txt`.
3. Search for the first directory in the unzipped files that contains a `callbacks.py`.
4. Copy this directory to our `agent_code`.
5. Run a single game with `self.train = False` against three `random_agents`.

We will then send you the script's output to the console including stack traces, the game logs, and the logs printed to your agent's `logs` folder.

We will test all submissions uploaded to the MaMpf Assignment "Final project submission test" by the test deadline 17.09.26 at 9pm. This gives you another weekend to fix any crashes before final agent submission.

9 Report and code repository

Put all your code into a public [Github](#) or [Bitbucket](#) repository and include the URL into your report. The tournament zip-file to be uploaded on MaMpf shall only contain the subdirectory of `agent_code` containing your fully trained best player.

Your report is due one week later. It should consist of about 4000 words per team member (and not much more, please), not counting title page etc. For legal reasons, do not use the university logo in your report. It should have the following structure (which is a standard for scientific work):

1. **Introduction.** Briefly state the problem you are trying to solve and explain why the task is worthwhile and challenging.
2. **Background.** Describe the task in more detail and give an overview of possible solution methods (here: different approaches to reinforcement learning that you considered).
3. **Project planning.** Describe how you organized the team work, planned your time to meet the deadline, distributed subtasks among the team members, and collaborated to produce the best possible results. If you use deep learning, also describe where and how you allocated the required hardware resources for training (e.g. Google Colab).
4. **Methods.** Describe how you specialized the general methods from the "background" section for the concrete task. Explain and justify crucial design choices, list the variants to be tested in the experiments, and design a methodology for systematic testing of agent performance

(e.g. define performance metrics). You may also describe approaches you tried and abandoned later, including the reasons. Include references to relevant literature. Recall that each team should implement at least two different agents.

5. **Training.** Describe your training process, including all tricks employed to speed it up (e.g. self play strategy, design of auxilliary rewards, prioritization of experience replay and so on).
6. **Experiments and Results.** Report experimental results (e.g. training progress diagrams, performance comparisons between your agents and the predefined ones), describe interesting observations, and discuss the difficulties you faced and how you overcame them. **This is the most important section of the report. It is critical that you systematically evaluate how changes in the design and implementation improve performance (or fail to do so) and which of your agents is ultimately the best.**
7. **Conclusion.** Summarize the findings of your report. In addition, give an outlook on how you would improve your agent if you had more time, and how we can improve the game setup for next year.

Following the standards of scientific publication and systematic experimentation will be crucial for your grade. If you are unsure about proper scientific writing, inform yourself in the internet or ask the tutors on Discord.

Make sure to be concise and clear. To judge your own writing, imagine that other students would like to replicate your results – your report should give them the information to do so in a form they can understand. **Mark each chapter or section with its main author, so that we can grade each team member's work individually – this is important for legal reasons.**